LAPORAN PRAKTIKUM MOBILE PROGRAMMING

Desain GUI, Navigasi, dan Integrasi API dengan JSON Serialization di Flutter

Nama : Faril Isra Albiansyah

NIM : 240605110087

Kelas : Mobile Programming Praktikum A

Tanggal : 17 – 09 – 2026

JURUSAN TEKNIK INFORMATIKA

FAKULTAS SAINS DAN TEKNOLOGI

UNIVERSITAS ISLAM NEGERI MAULANA MALIK IBRAHIM

MALANG

GANJIL 2025/2026

I. Tujuan

- Mendesain antarmuka (GUI) aplikasi Flutter yang interaktif dan responsif [cite: 13].
- Menerapkan sistem navigasi antarhalaman menggunakan mekanisme *named route*.
- Membuat model data dengan JSON *serialization* dan *deserialization* [cite: 13].
- Mengintegrasikan data dari API eksternal ke dalam aplikasi menggunakan paket http.
- Menampilkan data hasil permintaan API ke layar aplikasi dengan tata letak yang rapi dan dinamis.

II. Langkah Kerja

A. Langkah Langkah Pembuatan Aplikasi

- Membuat proyek Flutter baru bernama game_app dan menyusun arsitektur proyek dengan membuat folder model, view, dan viewmodel.
- Menambahkan dependensi paket http, json_annotation, readmore, build_runner, dan json_serializable pada file pubspec.yaml, lalu menjalankan perintah flutter pub get.
- Membuat kelas model data (game.dart dan detailgame.dart) dengan anotasi @JsonSerializable() untuk mendefinisikan struktur data dari API.
- Menjalankan perintah flutter pub run build_runner build di terminal untuk melakukan *code generation* (*file* .g.dart) yang mengotomatisasi proses konversi JSON ke objek Dart.
- Membuat berkas logika *viewmodel* (fetchgame.dart) yang berisi *method* asinkron fetchGames() dan fetchDataFromAPI() untuk mengeksekusi permintaan HTTP GET ke *endpoint* API FreeToGame.
- Membangun antarmuka (*view*) halaman Home menggunakan FutureBuilder, ListView.builder, dan Card untuk menyajikan data daftar game secara terstruktur.
- Membangun antarmuka (*view*) halaman Detail menggunakan FutureBuilder, Stack, SingleChildScrollView, serta ReadMoreText untuk

menampilkan spesifikasi rinci game, daftar *screenshot* yang dapat digulir horizontal, dan deskripsi teks.
- Mengatur konfigurasi *named route* terpusat pada file main.dart melalui properti routes.
- Mengimplementasikan navigasi pada kartu daftar game di halaman Home menggunakan Navigator.pushNamed() dengan membawa arguments berupa ID game, serta mengimplementasikan Navigator.pop() pada halaman Detail untuk fungsi tombol kembali.

B. Penjelasan Kode Program Utama Dalam pengembangan aplikasi ini, terdapat beberapa bagian kode utama yang menjadi inti dari fungsionalitas aplikasi [cite: 13]:

- **JSON Serialization pada Model (file `detailgame.dart`)**

Dart
```
@JsonSerializable()
class DetailGame {
  final int id;
  final String title;
  // ... deklarasi variabel lainnya

  factory DetailGame.fromJson(Map<String, dynamic> json) =>
_$DetailGameFromJson(json);
}
```

Penjelasan: Anotasi `@JsonSerializable()` digunakan untuk menandai kelas agar *build_runner* dapat membuatkan kode otomatis (file `.g.dart`) [cite: 13]. Fungsi `fromJson` bertugas melakukan *deserialization*, yaitu mengonversi data mentah JSON yang ditarik dari API menjadi objek Dart yang terstruktur dan mudah digunakan di dalam aplikasi.

- **Integrasi API dengan HTTP GET (file `fetchgame.dart`)**

Dart
```
Future<Map<String, dynamic>> fetchDataFromAPI(int idGame) async
{
  final response = await
http.get(Uri.parse('https://www.freetogame.com/api/game?id=$idG
ame'));
  if (response.statusCode == 200) {
    return json.decode(response.body);
  } else {
    throw Exception('Gagal mengambil data dari API');
  }
}
```

Penjelasan: Kode ini menggunakan paket `http` untuk melakukan *request* ke *server* luar secara asinkron (ditandai dengan `Future` dan `await`) `[cite: 13]`. Jika *server* merespons dengan sukses (kode 200), respons bertipe *string* akan diterjemahkan menjadi *Map* menggunakan `json.decode [cite: 13]`.

- **Pengelolaan UI Asinkron dengan FutureBuilder (file `home.dart` / `detail.dart`)**

Dart
```
FutureBuilder(
  future: fetchData(),
  builder: (context, snapshot) {
    if (snapshot.connectionState == ConnectionState.waiting) {
      return const Center(child: CircularProgressIndicator());
    } else if (snapshot.hasData) {
      // Menampilkan UI utama jika data berhasil dimuat
    }
  }
)
```

Penjelasan: `FutureBuilder` adalah *widget* krusial untuk menangani data yang membutuhkan waktu *loading* dari internet `[cite: 13]`. Kode ini mengecek *state* dari proses: jika masih `waiting`, maka akan memunculkan indikator *loading* memutar (`CircularProgressIndicator`); jika data sudah siap (`hasData`), maka UI utama akan dirender.

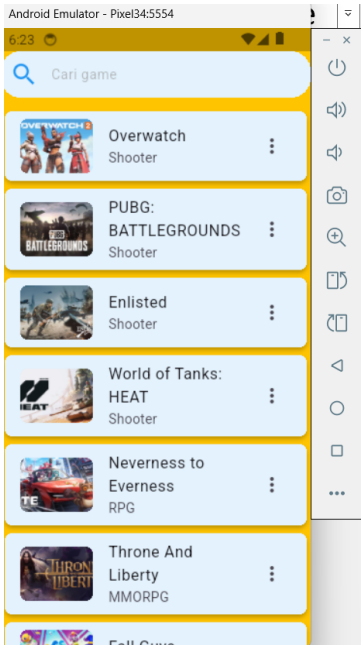
- **Navigasi Named Route dengan Argumen (file `home.dart` dan `main.dart`)**

Dart
```
// Pada Home (Pengirim):
Navigator.pushNamed(context, '/detail', arguments: game.id);

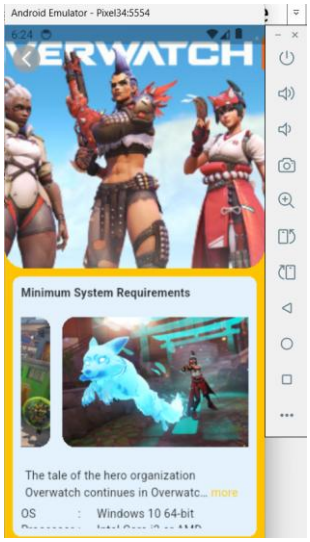
// Pada main.dart (Penerima):
'/detail': (context) {
  final gameId = ModalRoute.of(context)!.settings.arguments as
int;
  return Detail(gameTerpilih: gameId);
}
```

Penjelasan: Aplikasi menggunakan *Named Route* agar perpindahan antarlayar lebih rapi `[cite: 13]`. `Navigator.pushNamed()` digunakan untuk membuka rute `'/detail'` sekaligus membawa muatan data (*arguments*) berupa `game.id [cite: 13]`. Di `main.dart`, argumen tersebut ditangkap menggunakan `ModalRoute` dan diteruskan ke parameter halaman.

III. Screenshot Hasil



Gambar 1. *Tampilan Halaman Home pada aplikasi game_app. Menampilkan daftar game dinamis dari respons API yang disusun rapi menggunakan ListView.builder dan Card, serta dilengkapi dengan kolom pencarian (Search Bar) di bagian atas.*



Gambar 2. *Tampilan Halaman Detail yang dimuat setelah pengguna melakukan klik pada salah satu daftar game di Halaman Home. Halaman ini berhasil menangkap ID argumen, melakukan request API baru untuk detail spesifik, dan menampilkan cover, spesifikasi sistem, screenshot, serta deskripsi panjang game.*

IV. Kesimpulan

- Pemisahan struktur proyek (*modularisasi*) menjadi model, view, dan viewmodel sangat mendukung pengembangan aplikasi yang tertata, mudah dibaca, dan efisien untuk dipelihara.
- Penggunaan paket json_serializable mempermudah dan mempercepat konversi format data JSON yang diterima dari API ke dalam bentuk objek Dart secara otomatis.
- Widget FutureBuilder terbukti krusial dalam menangani aliran data asinkron dari API, karena memfasilitasi antarmuka untuk merespons status secara otomatis (menampilkan *loading* saat menunggu, *error message* jika gagal, dan menyajikan UI utama saat data telah siap).
- Pendekatan navigasi *named route* mempermudah pengelolaan perpindahan antarhalaman pada aplikasi. Fungsi Navigator.pushNamed() memungkinkan perpindahan rute sekaligus pengiriman parameter data

(arguments), sedangkan Navigator.pop() memberikan cara praktis untuk menghapus layar saat ini dan kembali ke halaman sebelumnya.